

■ GUÍA DE DEFENSA

Tienda PHP

Todo lo que necesitas saber para defender el proyecto

Explicado de forma sencilla · 2.º DAW

Este documento está pensado para que puedas repasar el proyecto antes de la defensa. Está escrito de forma sencilla, con analogías y ejemplos, para que lo entiendas bien y puedas explicarlo con tus propias palabras.

Índice

1. Qué es el proyecto y qué hace
2. Cómo está organizado el código (MVC)
3. Cómo llega una petición al código — Front Controller
4. La base de datos
5. Login y registro (clásico + Google OAuth)
6. La cesta de la compra
7. El pago con PayPal
8. El panel de administración
9. Seguridad — preguntas frecuentes
10. Preguntas típicas de defensa con respuestas
11. Glosario rápido

1. ¿Qué es el proyecto y qué hace?

Tienda PHP es una **tienda online completa hecha en PHP puro**, sin usar ningún framework como Laravel o Symfony. Eso significa que todo está hecho a mano: el sistema de rutas, la arquitectura, la gestión de sesiones, todo.

La tienda permite:

- Ver un catálogo de productos, filtrar por categoría y buscar por nombre.
- Registrarse (con confirmación por email) o iniciar sesión con Google.
- Añadir productos a una cesta de la compra.
- Pagar con PayPal (en modo sandbox, es decir, sin dinero real).
- Ver un panel de administración para gestionar productos, categorías y usuarios.

■ *Analogía: Es como Amazon pero hecha a mano desde cero para demostrar que sabes cómo funciona por dentro.*

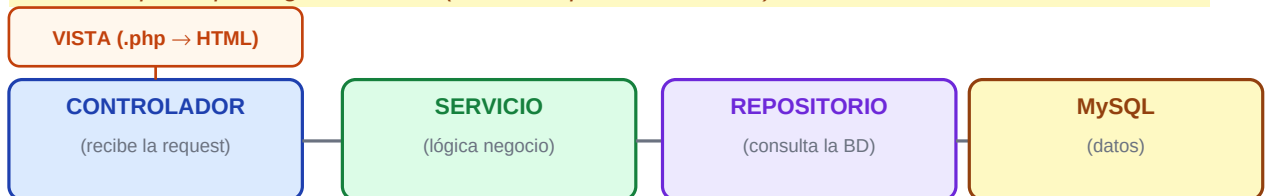
Tecnologías que usamos

Tecnología	¿Para qué sirve en el proyecto?
PHP 8.0+	El lenguaje principal. Todo el backend está en PHP.
MySQL	La base de datos donde guardamos productos, usuarios, pedidos...
PDO	La forma segura de hablar con MySQL desde PHP.
Composer	Gestor de dependencias. Como npm pero para PHP.
PHPMailer	Librería para enviar emails (confirmación de registro).
PayPal SDK	Para procesar pagos con PayPal Sandbox.
Google OAuth 2.0	Para que el usuario pueda hacer login con su cuenta de Google.
XAMPP	Servidor local (Apache + MySQL) para desarrollar en tu PC.

2. Cómo está organizado el código

El proyecto sigue una arquitectura **MVC** (Modelo–Vista–Controlador). Esto es una forma de organizar el código para que cada cosa esté en su sitio y no se mezcle todo.

■ *Analogía: Piensa en un restaurante: el **controlador** es el camarero (recibe tu pedido), el **servicio** es el jefe de cocina (decide cómo hacerlo), el **repositorio** es el cocinero (accede a la nevera/BD) y la **vista** es el plato que llega a la mesa (el HTML que ve el usuario).*



El controlador llama al servicio, que llama al repositorio, que consulta MySQL

Además del MVC clásico, hemos añadido dos capas extra que separan mejor las responsabilidades:

Capa	¿Qué hace?	Ejemplo en el proyecto
Controlador	Recibe la petición HTTP y decide qué hacer	AuthControlador, ProductoControlador...
Servicio	Contiene la lógica de negocio	UsuarioServicio hashea la clave y genera el token
Repositorio	Solo habla con la base de datos	UsuarioRepositorio hace el SELECT/INSERT a MySQL
Modelo	Representa una entidad de datos	Producto, Usuario, Pedido, Categoria
Vista	Genera el HTML que ve el usuario	Ficheros .php en src/Vistas/
Middleware	Filtra quién puede acceder a una ruta	AdminMiddleware bloquea el panel si no eres admin
Request	Valida los datos del formulario	RegistroRequest comprueba que el email es válido
Lib	Clases de ayuda reutilizables	Sesion, Cesta, Enrutador, GoogleOAuth...

3. ¿Cómo llega una petición al código? El Front Controller

Esta es probablemente la parte más importante de entender. Cuando escribes una URL como `/producto/detalle/5`, ¿cómo sabe PHP qué código ejecutar?



Paso 1 — El `.htaccess` redirige todo a `index.php`

El fichero `public/.htaccess` tiene una regla que le dice a Apache: "si alguien pide algo que no es un fichero real ni una carpeta real, mándalo a `index.php`". Esto hace que TODAS las URLs de la tienda pasen por un único punto de entrada.

■ La URL `/producto/detalle/5` NO es un fichero físico. El `.htaccess` la captura y la manda a `index.php`.

Paso 2 — `index.php` arranca la aplicación

`public/index.php` solo hace una cosa: incluir `src/init.php`. Ese fichero es el que prepara todo:

- Define constantes de rutas (RAIZ, APP, PUBLICO...).
- Carga el fichero `.env` con las configuraciones (BD, SMTP, PayPal...).
- Registra el autoloader para que PHP encuentre las clases automáticamente.
- Inicia la sesión PHP.
- Llama a `Rutas::registrar()` que define todas las rutas y despacha la petición.

Paso 3 — El Enrutador encuentra la ruta

`Lib/Enrutador.php` tiene una tabla de rutas registradas. Cada entrada asocia un método HTTP (GET o POST) + una URL → a un controlador. Cuando llega una petición, el enrutador busca en esa tabla.

```
// En Rutas.php están registradas TODAS las rutas:
Enrutador::agregar('GET', '/producto/detalle/:id', function ($id) {
    (new ProductoControlador())->detalle($id);
});
```

Si la URL es `/producto/detalle/5`, el enrutador detecta que encaja con el patrón `/producto/detalle/:id` y extrae el 5 como parámetro. Luego llama al controlador pasándole ese 5.

■ Analogía: El enrutador es como el recepcionista de un hotel: tú dices "quiero la habitación 5" y él sabe exactamente a qué puerta mandarte.

Paso 4 — El Controlador gestiona la petición

El controlador recibe los datos, los valida, llama al servicio que necesite y decide qué mostrar: o bien carga una vista con el resultado, o bien redirige al usuario a otra URL.

■ ¿Por qué usamos Front Controller en vez de tener un fichero PHP por cada página?

■ Con un fichero por página (producto.php, login.php...) el código se repite mucho y es difícil de mantener. Con un único punto de entrada podemos centralizar la seguridad, la sesión, el autoloader y las rutas. Es exactamente lo que hacen todos los frameworks modernos.

4. La base de datos

La base de datos se llama **tiendaphp** y tiene 5 tablas. Todas son InnoDB, lo que significa que soportan **claves foráneas** y **transacciones**.

Tabla	¿Qué guarda?	Relación importante
usuarios	Los usuarios registrados (nombre, email, clave hasheada, rol...)	1 usuario → N pedidos
categorias	Las categorías de productos (Electrónica, Ropa...)	1 categoría → N productos
productos	Los productos de la tienda con precio, stock e imagen	Pertenece a 1 categoría
pedidos	Cada compra realizada (importe, dirección, estado)	Pertenece a 1 usuario, tiene N líneas
lineas_pedido	Cada línea de un pedido (producto, cantidad, precio)	Pertenece a 1 pedido

¿Por qué `lineas_pedido` guarda el nombre y precio del producto?

Porque si mañana cambias el precio del producto, el pedido antiguo debe seguir reflejando el precio que se pagó en su momento. Por eso copiamos `nombre_producto` y `precio_unidad` en la línea del pedido en el momento de la compra.

■ **Nunca uses clave foránea directa al precio del producto en un pedido. El precio puede cambiar, el pedido no debe cambiar.**

Cómo nos conectamos a la BD — Patrón Singleton

Config/Conexion.php implementa el patrón Singleton: la primera vez que alguien llama a `Conexion::abrir()` se crea la conexión PDO. Las siguientes veces devuelve la misma conexión sin volver a conectarse.

■ *Analogía: Es como una llave maestra: la primera vez la sacas del cajón, el resto del tiempo usas la misma llave que ya tienes en la mano.*

¿Qué es PDO y por qué lo usamos?

PDO (PHP Data Objects) es una capa de abstracción para acceder a bases de datos. Lo importante es que usa **prepared statements** (consultas preparadas), que son la forma segura de pasar datos a MySQL.

```
// MAL (vulnerable a SQL Injection):
$sql = "SELECT * FROM usuarios WHERE email = '" . $email . "'";

// BIEN (prepared statement con PDO):
$stmt = $pdo->prepare("SELECT * FROM usuarios WHERE email = :email");
```

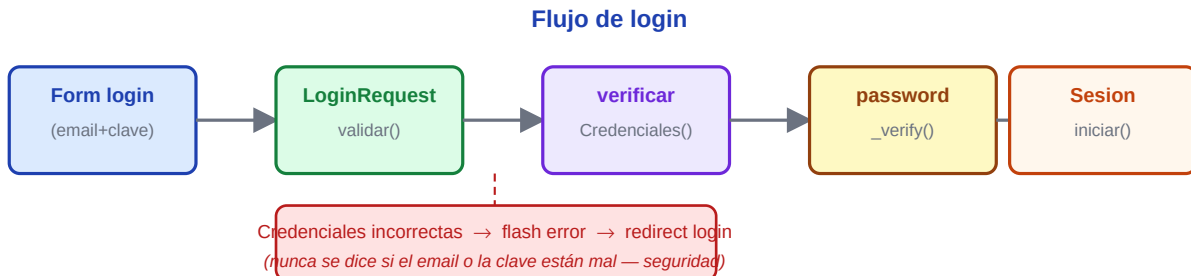
```
$stmt->execute([':email' => $email]);
```

■ ¿Qué es una SQL Injection y cómo la prevenimos?

■ Un ataque donde el usuario mete código SQL en un formulario para manipular la base de datos. Por ejemplo, en el campo email escribe "admin@x.com' OR '1'='1". Con PDO y prepared statements esto es imposible porque el dato nunca se interpreta como código SQL.

5. Cómo funciona el login y el registro

Registro de usuario — paso a paso



- El usuario rellena el formulario de registro y pulsa "Registrarse".
- **RegistroRequest::validar()** comprueba que los campos están bien: nombre no vacío, email válido, contraseñas coinciden, longitud mínima.
- Se comprueba que el email no exista ya en la BD.
- **UsuarioServicio::registrar()** genera un token aleatorio de 32 caracteres, hashea la contraseña con bcrypt e inserta el usuario con activado = 0.
- **EnvioMail** envía un email con un enlace de activación que incluye el token.
- Cuando el usuario hace clic en el enlace, se pone activado = 1 y se borra el token.

¿Cómo se guardan las contraseñas?

Nunca se guarda la contraseña en texto plano. Se usa **bcrypt** (función `password_hash()` de PHP). bcrypt transforma la contraseña en una cadena irreversible con un salt aleatorio incluido.

```
// Al registrar – guardamos el hash, nunca la contraseña:
password_hash($clave, PASSWORD_BCRYPT) // resultado: $2y$10$...

// Al hacer login – comparamos sin desencriptar:
password_verify($claveIntroducida, $hashGuardado) // true/false
```

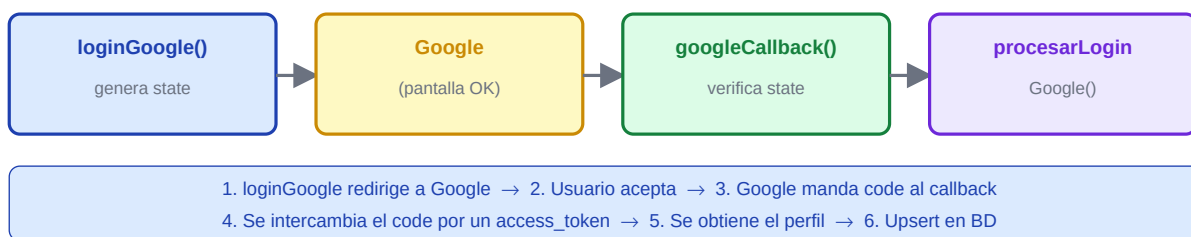
■ **bcrypt es un hash de una sola dirección. NO se puede descifrar. Para verificar, se hashea de nuevo y se comparan los hashes.**

Los mensajes flash

Cuando hay un error o un éxito, no podemos mostrar el mensaje directamente porque hacemos una redirección. Usamos **mensajes flash**: se guarda el mensaje en sesión, se redirige, y en la siguiente página se lee y se borra. Así el mensaje aparece solo una vez.

Login con Google OAuth 2.0

Flujo Google OAuth 2.0



- loginGoogle() genera un **state** aleatorio (protección anti-CSRF), lo guarda en sesión y redirige al usuario a la página de login de Google.
- Google muestra su pantalla de consentimiento. El usuario acepta.
- Google redirige al callback con un **code** y el **state**.
- Verificamos que el state recibido es igual al que guardamos (seguridad CSRF).
- Intercambiamos el code por un **access_token** haciendo una petición a Google.
- Con el access_token obtenemos el perfil del usuario (email, nombre, foto).
- Hacemos un **upsert**: si el usuario ya existe (mismo email o google_id) lo actualizamos; si no, lo creamos.
- Los usuarios de Google se crean directamente con activado = 1 porque Google ya verificó su email.

■ ¿Qué es el state en OAuth y para qué sirve?

- Es un token aleatorio que generamos nosotros antes de redirigir a Google. Google nos lo devuelve en el callback. Si el state no coincide con el que guardamos en sesión, rechazamos la petición. Esto evita ataques CSRF donde alguien engañe a un usuario para que complete el flujo OAuth sin querer.

6. La cesta de la compra

La cesta se guarda en la **sesión PHP** como un array simple: [id_producto => cantidad]. No hay tabla en BD para la cesta, todo está en memoria de sesión.

■ *Analogía: Es como un carrito físico en un supermercado: lo llevas contigo mientras haces la compra, pero si te vas sin pagar se queda vacío. La sesión PHP dura mientras el navegador esté abierto.*

Cesta de invitado vs cesta de usuario

La clave en sesión cambia según si estás logueado o no:

Estado	Clave en \$_SESSION	Qué pasa al hacer login
Sin login (invitado)	cesta_invitado	Se fusiona con la cesta del usuario
Con login	cesta_u{id} (p.ej. cesta_u42)	Se usa directamente

Si añades productos sin estar logueado y luego haces login, el método **Cesta::migrarAlLogin()** fusiona ambas cestas sumando las unidades de cada producto.

Validación de stock

Cada vez que se añade un producto, **Cesta::meterProducto()** consulta el stock real en la BD en ese momento. Si el total que quieres añadir supera el stock disponible, se devuelve un error y no se añade.

■ ¿Por qué no guardamos la cesta en la base de datos?

■ Para este proyecto la sesión es suficiente y más sencillo. En una tienda real con millones de usuarios se guardaría en BD (o Redis) para que persista si el usuario cierra el navegador. Es una mejora pendiente del proyecto.

7. El pago con PayPal

Flujo de pago PayPal



Usamos la **API REST de PayPal Sandbox**. El sandbox es un entorno de pruebas donde se simula el pago sin dinero real. El flujo tiene 3 fases:

Fase 1 — Crear la orden

Cuando el usuario pulsa "Pagar", el servidor pide un **access token** a PayPal con las credenciales (Client ID + Secret). Con ese token crea una **orden de intención CAPTURE** con el importe total. PayPal devuelve un enlace de aprobación al que redirigimos al usuario.

Fase 2 — El usuario aprueba en PayPal

El usuario ve la pantalla de PayPal, inicia sesión con su cuenta sandbox y aprueba el pago. PayPal redirige al usuario a nuestra URL de retorno (/pago/exito) con el ID de la orden.

Fase 3 — Capturar el pago y crear el pedido

En /pago/exito, verificamos que el ID de la orden coincide con el que guardamos en sesión. Luego llamamos a **capturarOrden()**: si el status es COMPLETED, creamos el pedido en BD dentro de una **transacción** y vaciamos la cesta.

■■ La transacción PDO garantiza que si algo falla a mitad (luz, error...), el pedido no queda a medias: o se crea entero o no se crea nada.

■ ¿Por qué verificamos que el token de PayPal coincide con el de la sesión?

■ Para evitar que alguien manipule la URL y llegue a /pago/exito con un token falso o reutilizado. Solo si el token recibido es el mismo que guardamos al crear la orden confiamos en que el pago es legítimo.

8. El panel de administración

El panel está en la ruta /admin y solo pueden acceder los usuarios con rol admin. Esto lo controla **AdminMiddleware**, que es lo primero que se ejecuta en cualquier ruta del panel.

Sección	Operaciones
Productos	Ver lista · Crear nuevo · Editar · Eliminar · Subir imagen
Categorías	Ver lista · Crear nueva · Eliminar (solo si no tiene productos)
Usuarios	Ver lista con rol y estado · Eliminar usuario

Subida de imágenes

Cuando se crea o edita un producto, se puede subir una imagen. El controlador valida que el fichero es una imagen (por tipo MIME), genera un nombre único con `time()` + `uniqid()` y lo guarda en `public/uploads/imagenes/`.

■ ¿Por qué generamos un nombre único para la imagen en vez de usar el nombre original?

■ Para evitar que dos productos con la misma imagen sobrescriban el fichero, y para evitar que alguien suba un fichero con un nombre malicioso como `"../../config/.env"`.

9. Seguridad — preguntas frecuentes

■ ¿Cómo prevenimos XSS (inyección de HTML/JS)?

■ Usando `htmlspecialchars()` en todos los datos que vienen del usuario antes de insertarlos en BD o de mostrarlos en pantalla. Esto convierte `<` en `<` y `>` en `>`, por lo que el navegador no ejecuta el código.

■ ¿Cómo prevenimos SQL Injection?

■ Con PDO y prepared statements. Los parámetros nunca se concatenan directamente en el SQL, sino que se pasan como parámetros separados. MySQL los trata como datos, nunca como código.

■ ¿Por qué los usuarios nuevos tienen `activado=0`?

■ Para evitar que alguien se registre con un email que no es suyo. Mientras no confirme el email mediante el enlace con token, no puede hacer login.

■ ¿Qué pasa si alguien intenta acceder al panel de admin sin ser admin?

■ `AdminMiddleware` llama a `Sesion::exigirAdmin()`, que comprueba si hay sesión y si el rol es "admin". Si no, redirige al inicio con un mensaje de error. El controlador nunca llega a ejecutarse.

■ ¿Por qué usamos ENUM en los campos rol y estado?

■ Para que MySQL solo permita valores válidos. El rol solo puede ser "cliente" o "admin". El estado del pedido solo puede ser "pendiente", "enviado", "entregado" o "cancelado". Si se intenta insertar otro valor, MySQL lo rechaza.

■ ¿El .env se sube a GitHub?

■ No, está en el .gitignore. El .env contiene contraseñas de BD, credenciales SMTP y claves de PayPal que nunca deben ser públicas. En su lugar se sube un .env.example con los nombres de las variables pero sin valores.

10. Preguntas típicas de defensa

■ ¿Por qué no usaste un framework como Laravel?

■ El objetivo era demostrar que entiendo cómo funciona la arquitectura MVC por dentro, no solo usar las herramientas que lo hacen automático. Construir el enrutador, el sistema de sesiones y las capas desde cero demuestra que sé lo que hace Laravel "bajo el capó".

■ ¿Qué es el patrón Front Controller?

■ Es tener un único punto de entrada para todas las peticiones (index.php). El .htaccess redirige todo ahí, y el enrutador decide qué controlador ejecutar según la URL y el método HTTP. Así centralizamos la seguridad, la sesión y las rutas.

■ ¿Para qué sirve Composer en este proyecto?

■ Para dos cosas: gestionar las dependencias externas (PHPMailer y PayPal SDK) y generar el autoloader PSR-4 que permite que PHP encuentre las clases automáticamente según su namespace sin tener que hacer require() a mano.

■ ¿Por qué separas Servicios de Repositorios?

■ Para separar la lógica de negocio (qué hace el sistema) del acceso a datos (cómo lo guarda). Si mañana cambiamos de MySQL a PostgreSQL, solo tocamos los Repositorios. Si cambia una regla de negocio, solo tocamos el Servicio.

■ ¿Cómo funciona el autoloader?

■ En init.php registramos un spl_autoload_register() que mapea cada namespace a una carpeta. Cuando PHP ve "new Servicios\UsuarioServicio()", busca en src/Servicios/UsuarioServicio.php y lo carga automáticamente.

■ ¿Cómo garantizas que el stock no queda descontado si el pago falla?

■ El stock solo se descuenta dentro de la transacción PDO que crea el pedido. Esta transacción solo se ejecuta después de que PayPal confirme el pago con status COMPLETED. Si algo falla, se hace rollback() y no cambia nada.

■ ¿Qué es un mensaje flash?

■ Es un mensaje de éxito o error que se guarda en sesión, se muestra una sola vez en la siguiente página y luego se borra. Se usa después de redirecciones para mostrar "Registro completado" o "Credenciales incorrectas".

■ ¿Qué diferencia hay entre GET y POST en tus rutas?

■ GET es para obtener datos (ver catálogo, ver detalle). POST es para enviar datos que modifican algo (login, registro, añadir producto). Las rutas del enrutador especifican el método, por lo que la misma URL puede tener comportamiento distinto según si es GET o POST.

■ ¿Por qué los usuarios de Google se crean con activado=1 directamente?

■ Porque Google ya ha verificado que ese email pertenece a esa persona. No tiene sentido mandar otro email de confirmación si Google ya lo hizo.

■ ¿Qué mejoras añadirías si tuvieras más tiempo?

■ Añadir tokens CSRF en los formularios POST, historial de pedidos para el cliente, recuperación de contraseña por email, paginación en el panel admin, tests unitarios con PHPUnit para los Servicios.

11. Glosario rápido

Término	Qué significa en este proyecto
MVC	Model-View-Controller. Forma de organizar el código en capas.
PDO	PHP Data Objects. Capa para hablar con MySQL de forma segura.
Prepared statement	Consulta SQL parametrizada que previene SQL Injection.
bcrypt	Algoritmo de hash para contraseñas. Irreversible e incluye salt.
Salt	Dato aleatorio que se mezcla con la contraseña antes de hashear.
OAuth 2.0	Protocolo para autenticar con un servicio externo (Google).
CSRF	Ataque que hace que el usuario ejecute acciones sin saberlo.
State (OAuth)	Token aleatorio que previene ataques CSRF en el flujo OAuth.
Front Controller	Patrón: una sola entrada (index.php) para todas las peticiones.
Singleton	Patrón: una sola instancia de una clase (Conexion PDO).
Upsert	INSERT si no existe, UPDATE si ya existe.
Flash message	Mensaje de sesión que se muestra una vez y desaparece.
Middleware	Código que se ejecuta antes del controlador (comprueba permisos).
Sandbox (PayPal)	Entorno de pruebas de PayPal sin dinero real.
Autoload	Carga automática de clases PHP sin hacer require() manualmente.
Namespace	Espacio de nombres PHP para organizar clases y evitar conflictos.
Transaction (BD)	Bloque de operaciones SQL que o se ejecutan todas o ninguna.
Rollback	Deshacer una transacción si algo falla.
InnoDB	Motor de MySQL que soporta claves foráneas y transacciones.
ENUM	Tipo de dato MySQL que solo acepta valores de una lista cerrada.